

VM Reporting – Projektdokumentation

Projektziel

Ziel des Projektes ist die Entwicklung einer modularen Python-Anwendung zur automatisierten Erfassung, Verarbeitung, Bewertung und Berichterstellung von VMware-vCenter-Metriken.

Die Anwendung sammelt regelmäßig historische Leistungsdaten virtueller Maschinen, speichert Rohdaten in einer relationalen Datenbank, aggregiert diese monatlich und erzeugt HTML-Reports zur Bewertung der Ressourcenauslastung.

Erfasste Metriken:

- CPU-Auslastung
- Arbeitsspeicherauslastung
- Storage-Auslastung

Die Anwendung soll:

- unter Windows ausführbar sein
- automatisierbar sein
- robust gegenüber fehlenden Metriken arbeiten
- modular erweiterbar bleiben
- fachlich nachvollziehbare Reports erzeugen
- auch ohne vCenter-Zugriff im Demo-Modus präsentierbar sein

Architekturübersicht

Die Anwendung verwendet eine mehrschichtige Architektur.

```
CLI
→ Services
→ Technical Modules
→ Repositories
→ Database
```

Dadurch werden:

- Präsentationslogik
- Geschäftslogik
- technische Integrationen
- Datenzugriff

sauber voneinander getrennt.

Die CLI enthält nur die Benutzersteuerung. Die eigentlichen Abläufe werden in Services gekapselt. Technische Module wie vCenter-Client, Aggregation, Evaluation und Reporting übernehmen klar abgegrenzte Aufgaben.

Der Datenbankzugriff erfolgt über Repository-Funktionen.

Projektstruktur

```
app/
├── cli/
│   ├── commands.py
│   └── debug_commands.py
├── collectors/
│   └── metric_collector.py
├── config/
│   ├── settings.py
│   ├── threshold_loader.py
│   └── thresholds.yaml
├── database/
│   ├── connection.py
│   ├── models.py
│   └── repositories.py
├── processing/
│   ├── aggregation.py
│   ├── evaluation.py
│   └── storage_usage.py
├── reporting/
│   ├── html_report.py
│   ├── templates/
│   │   └── monthly_report.html.j2
│   └── static/
│       ├── report.css
│       └── report.js
├── services/
│   ├── aggregation_service.py
│   ├── collection_service.py
│   ├── demo_data_service.py
│   ├── reporting_service.py
│   └── workflow_service.py
├── utils/
│   ├── date_utils.py
│   └── logging_config.py
└── vcenter/
    ├── factory.py
    ├── pyvmomi_client.py
    └── archive/
        └── rest_client.py
```

main.py

Datenmodell

Das Datenmodell trennt bewusst zwischen:

- VM-Stammdaten
- Sammelläufen
- Rohmetriken
- Monatsaggregaten
- Bewertungsergebnissen

Diese Trennung ermöglicht Nachvollziehbarkeit, Wiederholbarkeit der Aggregation und eine spätere Erweiterung des Reportings.

virtual_machines

Speichert Stammdaten virtueller Maschinen.

Feld	Typ	Beschreibung
id	Integer	Interne Datenbank-ID, Primärschlüssel
moid	String(100)	VMware Managed Object ID, eindeutig
name	String(255)	Anzeigename der VM
created_at	DateTime	Erstellungszeitpunkt des Datensatzes

Beziehungen:

Beziehung	Ziel	Beschreibung
metric_records	MetricRecord	Rohmetriken der VM
monthly_aggregates	MonthlyAggregate	Monatsaggregate der VM

Die interne ID wird als Primärschlüssel verwendet, obwohl die VMware-MOID eindeutig ist. Dadurch bleibt die Datenbank von externen Identifikatoren entkoppelt und Beziehungen können effizient über Integer-Fremdschlüssel aufgebaut werden.

collection_runs

Dokumentiert jeden Sammellauf.

Feld	Typ	Beschreibung
id	Integer	Primärschlüssel

Feld	Typ	Beschreibung
started_at	DateTime	Startzeit des CollectionRuns
finished_at	DateTime, nullable	Endzeit des CollectionRuns
period_start	DateTime	Beginn des abgefragten Metrikzeitraums
period_end	DateTime	Ende des abgefragten Metrikzeitraums
status	String(20)	Status des Runs, z. B. SUCCESS oder FAILED
processed_vm_count	Integer	Anzahl verarbeiteter VMs
created_metric_record_count	Integer	Anzahl erzeugter MetricRecords
error_message	Text, nullable	Fehlermeldung bei Fehlern

Beziehungen:

Beziehung	Ziel	Beschreibung
metric_records	MetricRecord	Rohdaten, die in diesem Lauf erzeugt wurden

Wichtig ist die Trennung zwischen **started_at/finished_at** und **period_start/period_end**.

started_at und **finished_at** beschreiben, wann der Lauf technisch ausgeführt wurde. **period_start** und **period_end** beschreiben dagegen, für welchen Zeitraum vCenter-Metriken abgefragt wurden.

metric_records

Speichert Rohmetriken einzelner Messpunkte.

Feld	Typ	Beschreibung
id	Integer	Primärschlüssel
vm_id	Integer	Fremdschlüssel auf virtual_machines.id
collection_run_id	Integer	Fremdschlüssel auf collection_runs.id
timestamp	DateTime	Messzeitpunkt
cpu_usage_percent	Float, nullable	CPU-Auslastung in Prozent
memory_usage_percent	Float, nullable	Arbeitsspeicherauslastung in Prozent
storage_usage_percent	Float, nullable	Storage-Auslastung in Prozent
power_state	String(20)	Power-State der VM zum Zeitpunkt des Sammellaufs

Beziehungen:

Beziehung	Ziel	Beschreibung
virtual_machine	VirtualMachine	Zugehörige VM

Beziehung	Ziel	Beschreibung
collection_run	CollectionRun	Zugehöriger Sammellauf

CPU-, RAM- und Storgewerte sind nullable. Dadurch bleibt die Anwendung robust gegenüber:

- fehlenden VMware-Metriken
- ausgeschalteten VMs
- Templates
- fehlenden VMware Tools
- unvollständigen Gastinformationen
- VMs ohne verfügbare Gast-Dateisystemdaten

Der **timestamp** bleibt fachlich verpflichtend. Ein MetricRecord ohne Messzeitpunkt wäre fachlich nicht aussagekräftig.

monthly_aggregates

Speichert aggregierte Monatswerte pro VM und Monat.

Feld	Typ	Beschreibung
id	Integer	Primärschlüssel
vm_id	Integer	Fremdschlüssel auf virtual_machines.id
month	String(7)	Monat im Format YYYY-MM
metric_record_count	Integer	Anzahl berücksichtigter Rohdatensätze
cpu_avg_percent	Float, nullable	Durchschnittliche CPU-Auslastung
cpu_min_percent	Float, nullable	Minimale CPU-Auslastung
cpu_max_percent	Float, nullable	Maximale CPU-Auslastung
memory_avg_percent	Float, nullable	Durchschnittliche RAM-Auslastung
memory_min_percent	Float, nullable	Minimale RAM-Auslastung
memory_max_percent	Float, nullable	Maximale RAM-Auslastung
storage_avg_percent	Float, nullable	Durchschnittliche Storage-Auslastung
storage_min_percent	Float, nullable	Minimale Storage-Auslastung
storage_max_percent	Float, nullable	Maximale Storage-Auslastung
created_at	DateTime	Erstellungszeitpunkt
updated_at	DateTime, nullable	Aktualisierungszeitpunkt

Constraints:

Constraint	Beschreibung
uq_monthly_aggregate_vm_month	Pro VM und Monat darf nur ein Monatsaggregat existieren

Beziehungen:

Beziehung	Ziel	Beschreibung
virtual_machine	VirtualMachine	Zugehörige VM
evaluation_result	EvaluationResult	Bewertung des Monatsaggregats

Alle Aggregatwerte sind nullable. Wenn für eine Metrik keine verwertbaren Rohwerte vorhanden sind, bleibt der jeweilige Aggregatwert **NULL**.

Die Felder **created_at** und **updated_at** dienen der Nachvollziehbarkeit späterer Änderungen oder Neuberechnungen von Aggregaten. **updated_at** ist bereits für zukünftige Reaggregationen und Neubewertungen vorbereitet.

evaluation_results

Speichert Bewertungsergebnisse zu Monatsaggregaten.

Feld	Typ	Beschreibung
id	Integer	Primärschlüssel
monthly_aggregate_id	Integer	Fremdschlüssel auf monthly_aggregates.id
overall_status	String(20)	Gesamtbewertung
cpu_status	String(20), nullable	CPU-Bewertung
memory_status	String(20), nullable	RAM-Bewertung
storage_status	String(20), nullable	Storage-Bewertung
note	Text, nullable	Optionale Notiz
created_at	DateTime	Erstellungszeitpunkt
updated_at	DateTime, nullable	Aktualisierungszeitpunkt

Constraints:

Constraint	Beschreibung
uq_evaluation_result_monthly_aggregate	Pro Monatsaggregat darf nur ein Bewertungsergebnis existieren

Beziehungen:

Beziehung	Ziel	Beschreibung
monthly_aggregate	MonthlyAggregate	Zugehöriges Monatsaggregat

Statuswerte:

- UNDERUTILIZED
- NORMAL
- WARNING
- CRITICAL
- NO_DATA

Einzelstatuswerte können nullable sein, wenn für die jeweilige Metrik kein bewertbarer Wert vorhanden ist.

Auch hier dienen `created_at` und `updated_at` der Nachvollziehbarkeit späterer Neubewertungen. Das Feld `updated_at` ist aktuell vorbereitet, wird jedoch noch nicht aktiv verwendet.

vCenter-Anbindung

Die VMware-Kommunikation erfolgt über pyVmomi.

PyVmomiClient

Die Klasse `PyVmomiClient` kapselt:

- Verbindungsaufbau zu vCenter
- Trennung der Verbindung
- Zugriff auf das vCenter Inventory
- Abfrage aller virtuellen Maschinen
- Abfrage verfügbarer Performance Counter
- Ermittlung konkreter Counter-IDs
- Abfrage historischer Performancewerte
- Abfrage von Gast-Dateisystem-Rohdaten

Der Client ist damit die technische Integrationsschicht zwischen Anwendung und VMware vCenter.

VM-Abfrage

Die VMs werden über eine ContainerView aus dem vCenter Inventory geladen.

Dadurch können alle virtuellen Maschinen unterhalb des Root-Folders abgefragt werden. Der Client gibt echte `vim.VirtualMachine`-Objekte zurück, die anschließend im Collector weiterverarbeitet werden.

Performance Counter

CPU- und RAM-Werte werden über den VMware PerformanceManager abgefragt. vCenter stellt Performancewerte nicht direkt über sprechende Namen bereit, sondern über interne Counter-IDs.

Deshalb enthält der Client Methoden zur Counter-Ermittlung:

Methode	Zweck
<code>get_performance_counters</code>	Lädt alle verfügbaren Performance Counter aus vCenter
<code>find_counter_id</code>	Ermittelt die konkrete Counter-ID anhand von Gruppe, Name und Rollup
<code>get_counter_unit</code>	Ermittelt die Einheit des Counters, z. B. Prozent

Verwendete Counter:

Metrik	Gruppe	Name	Rollup
CPU	cpu	usage	average
RAM	mem	usage	average

Die Counter-Abfrage ist notwendig, weil `QueryPerf` nicht mit den Namen `cpu.usage.average` oder `mem.usage.average` arbeitet, sondern eine konkrete `counterId` benötigt. Diese ID kann je nach vCenter-Umgebung variieren und wird deshalb zur Laufzeit aus den verfügbaren Countern ermittelt.

Die Methode `get_performance_counters()` ist somit keine eigentliche Fachfunktion, sondern eine technische Hilfsfunktion für die robuste Ermittlung der benötigten vCenter-Counter.

Historische CPU- und RAM-Metriken

Die historischen Werte werden über `QueryPerf` abgefragt.

Dabei wird ein Zeitraum definiert:

```
start_time
end_time
```

und ein Intervall:

```
intervalId = 300
```

Der Parameter `intervalId` gibt in vCenter die gewünschte Auflösung der historischen Performancewerte an. Der Wert `300` steht für ein Fünf-Minuten-Intervall.

Für Prozentwerte liefert vCenter Rohwerte in Hundertstel-Prozent. Deshalb werden Werte mit der Einheit `percent` durch 100 geteilt.

Beispiel:

```
vCenter-Rohwert: 243
normalisierter Wert: 2.43 %
```

Storage-Auslastung

Die Storage-Auslastung wird nicht über den VMware PerformanceManager ermittelt.

Stattdessen wird die Gast-Dateisystembelegung verwendet:

```
vm.guest.disk
```

Dadurch entsteht eine realistischere Auslastung innerhalb des Betriebssystems.

Der PyVmomiClient liefert dabei nur normalisierte Rohdaten:

```
disk_path  
capacity  
free_space  
filesystem_type
```

Die eigentliche Berechnung erfolgt in:

```
app/processing/storage_usage.py
```

Berechnung:

```
used_percent =  
(sum(capacity) - sum(freeSpace)) / sum(capacity) * 100
```

Die Werte aller Laufwerke werden summiert.

Vorteile:

- reale Gastbelegung
- keine Thin-Provisioning-Verfälschung
- keine Werte über 100 %
- verständlichere Reports

Einschränkungen:

- VMware Tools erforderlich
- Gastinformationen müssen verfügbar sein
- wenn keine Gast-Dateisystemdaten vorhanden sind, bleibt der Storgewert **None**

Metriksammlung

Die Metriksammlung erfolgt über den Collector.

Für Storgewerte ruft der Collector zunächst die Guest-Disk-Rohdaten über den vCenter-Client ab und übergibt diese anschließend an `calculate_storage_usage_percent()` aus `storage_usage.py`.

CPU- und RAM-Werte werden zunächst als separate Zeitreihen geladen. Der Collector verwendet die CPU-Zeitreihe als führende Zeitbasis und ergänzt passende RAM-Werte anhand identischer Zeitstempel.

Dadurch entstehen konsistente MetricRecords mit gemeinsamem Timestamp für CPU- und RAM-Werte.

Ablauf:

```
collect-metrics / daily-workflow
→ collection_service
→ metric_collector
→ PyVmomiClient
→ repositories
→ database
```

Der Collector:

1. verbindet sich mit vCenter
2. lädt alle VMs
3. definiert den Abfragezeitraum
4. erstellt einen CollectionRun
5. sammelt CPU-, RAM- und Storgewerte
6. speichert MetricRecords
7. beendet den CollectionRun mit Status und Zählwerten
8. trennt die vCenter-Verbindung

Aggregation

Die Aggregation erfolgt in:

```
app/processing/aggregation.py
```

Berechnet werden:

- Durchschnitt
- Minimum
- Maximum

Die Aggregation ist vollständig None-sicher.

Fehlende Werte werden ignoriert.

Beispiel:

```
[10, 20, None, 30]  
→ Durchschnitt = 20
```

Wenn keine gültigen Werte vorhanden sind:

```
None
```

Bewertungssystem

Die Bewertung erfolgt in:

```
app/processing/evaluation.py
```

Thresholds werden extern geladen.

thresholds.yaml

```
cpu:  
  underutilized_below: 20  
  warning_above: 70  
  critical_above: 90
```

Dies ermöglicht:

- fachliche Anpassung ohne Codeänderung
- bessere Wartbarkeit
- klare Trennung von Logik und Konfiguration

Overall-Status

Die Gesamtbewertung wird aus Einzelbewertungen bestimmt.

Priorität:

```
CRITICAL  
→ WARNING  
→ UNDERUTILIZED  
→ NORMAL
```

Fehlende Einzelmetriken werden ignoriert.

Dadurch werden VMs mit:

```
CPU = UNDERUTILIZED  
RAM = UNDERUTILIZED  
Storage = NORMAL
```

insgesamt als:

```
NORMAL
```

bewertet.

Service-Schicht

Die Services kapseln Geschäftslogik.

`collection_service.py`

Startet die Metrikerfassung.

```
CLI  
→ collection_service  
→ metric_collector
```

`aggregation_service.py`

Führt Monatsaggregation und Bewertung durch.

```
CLI  
→ aggregation_service  
→ aggregation/evaluation  
→ repositories
```

`reporting_service.py`

Erzeugt HTML-Reports.

```
CLI  
→ reporting_service  
→ html_report
```

Zusätzlich ermittelt der Reporting-Service bekannte VMs ohne Metrikdaten für den betrachteten Monat. Dadurch kann der Report ausweisen, welche Inventory-VMs im Monatsreport nicht in der Haupttabelle erscheinen.

workflow_service.py

Orchestriert Tages- und Monatsworkflows.

Tagesworkflow:

```
daily-workflow
├─ collect metrics
```

Monatsworkflow:

```
monthly-workflow
├─ determine previous month
├─ aggregate
├─ check for data
├─ report
```

Dadurch entstehen keine verschachtelten CLI-Aufrufe.

HTML-Reporting

Die Reports werden erzeugt in:

```
output/reports/
```

Dateiname:

```
report_YYYY-MM.html
```

Templatebasierte Report-Erzeugung

Die HTML-Reports werden templatebasiert mit Jinja2 erzeugt.

Die Reportlogik ist in drei Bereiche getrennt:

```
app/reporting/
├─ html_report.py
├─ templates/
│   └─ monthly_report.html.j2
```

```
└─ static/
   └─ report.css
   └─ report.js
```

Verantwortlichkeiten

Datei	Aufgabe
html_report.py	bereitet Reportdaten auf, lädt CSS/JavaScript und rendert das Jinja2-Template
monthly_report.html.j2	definiert die HTML-Struktur des Reports
report.css	enthält das Styling des Reports
report.js	enthält die clientseitige Tabellensortierung

Dadurch enthält die Python-Datei keine großen HTML-, CSS- oder JavaScript-Blöcke mehr. Die Report-Erzeugung ist dadurch besser wartbar und klarer getrennt.

Der erzeugte Report bleibt weiterhin eine einzelne portable HTML-Datei. CSS und JavaScript werden beim Rendern in die HTML-Datei eingebettet.

Report-Inhalt

Der HTML-Report enthält mehrere fachliche Bereiche.

Report-Metadaten

Der Report zeigt:

- ausgewerteten Monat
- Erstellungszeitpunkt
- Anzahl bekannter Inventory-VMs
- Anzahl VMs mit Metrikdaten
- Anzahl VMs ohne Metrikdaten
- Anzahl kritischer bzw. warnender Systeme

Statuslegende

Die Statuslegende erklärt die verwendeten Statusfarben.

Inventory Summary

Der Report weist aus:

- Anzahl aller bekannten VMs im Inventory
- Anzahl der VMs mit Metrikdaten
- Anzahl der VMs ohne Metrikdaten

Dadurch wird transparent, wenn VMs zwar bekannt sind, für den betrachteten Zeitraum aber keine verwertbaren Metriken vorliegen.

Status Summary Cards

Der Report enthält farbliche Statuskarten für:

- CRITICAL
- WARNING
- NORMAL
- UNDERUTILIZED
- NO_DATA

Top Consumers

Anzeige der höchsten Verbraucher für:

- CPU
- RAM
- Storage

Mit farblicher Statusmarkierung.

Haupttabelle

Enthält:

- VM-Name
- Anzahl berücksichtigter Rohdatensätze
- durchschnittliche CPU-Auslastung
- durchschnittliche RAM-Auslastung
- durchschnittliche Storage-Auslastung
- farbliche Statusmarkierung je Metrik
- Overall-Status

VMs ohne Metrikdaten

VMs ohne Metrikdaten werden in einem eigenen Abschnitt aufgeführt.

Das verhindert, dass bekannte Systeme im Report stillschweigend fehlen.

Tabellenfunktionen

Die Haupttabelle unterstützt:

- clientseitige Sortierung
- Klick auf Tabellenkopf
- aufsteigende/absteigende Sortierung
- visuelle Dreiecke zur Sortierrichtung

Die Sortierlogik ist ausgelagert in:

```
app/reporting/static/report.js
```

Farbsystem

Verwendete Statusfarben:

Status	Farbe
UNDERUTILIZED	Blau
NORMAL	Grün
WARNING	Orange
CRITICAL	Rot
NO_DATA	Grau

Die Reports enthalten:

- Statuslegende
- farbige Statuspunkte
- farbige Top-Consumer-Markierungen
- farbige Statuskarten

Demo-Modus

Für Präsentationen ohne Zugriff auf ein echtes VMware-vCenter gibt es einen Demo-Modus.

Der Demo-Modus verwendet:

```
.env.demo
```

und trennt Demo-Daten von echten Daten.

```
Produktiv:
data/vm_reporting.db
output/reports/

Demo:
data/demo_vm_reporting.db
output/demo-reports/
```

Demo-Daten werden über folgenden Command erzeugt:


```
python -m app.main seed-demo-data 2026-05
```

Der Demo-Modus enthält synthetische VMs mit unterschiedlichen Auslastungszuständen, darunter:

- unterausgelastete VM
- normale VM
- VM mit kritischer Storage-Auslastung
- VM mit Warning-Status
- VM ohne Storage-Daten
- VM ohne Metrikdaten

Dadurch kann der Prüfer die Aggregation, Bewertung und Report-Erzeugung ohne vCenter-Zugriff nachvollziehen.

Debug-Commands

Unterkommandos:

```
python -m app.main debug ...
```

show-aggregates

Zeigt aggregierte Monatsdaten.

Beispiel:

```
python -m app.main debug show-aggregates 2026-05
```

show-evaluations

Zeigt Bewertungsergebnisse.

Beispiel:

```
python -m app.main debug show-evaluations 2026-05
```

Mit Statusfilter:

```
python -m app.main debug show-evaluations 2026-05 --status CRITICAL
```

Automatisierungskonzept

Tägliche Datensammlung

Geplanter Ablauf:

```
daily-workflow  
→ collect metrics
```

Jeder Run sammelt historische Werte eines definierten Zeitfensters.

Die Ausführung kann z. B. über die Windows-Aufgabenplanung erfolgen.

Monatlicher Workflow

```
monthly-workflow  
├─ previous month  
├─ aggregation  
├─ evaluation  
└─ report generation
```

Nur bei vorhandenen Daten wird ein Report erzeugt.

Fehlerrobustheit

Die Anwendung behandelt:

- fehlende Gastinformationen
- fehlende Storgewerte
- fehlende CPU/RAM-Metriken
- ausgeschaltete Systeme
- teilweise verfügbare Metriken
- VMs ohne Metrikdaten

ohne Abbruch des Gesamtlaufs.

Logging

Die Anwendung verwendet zentrales Logging.

Logdatei:

```
logs/vm_reporting.log
```

Geloggte Ereignisse:

- vCenter-Verbindungsaufbau
- VM-Ladevorgänge
- CollectionRuns
- Anzahl erzeugter MetricRecords
- Aggregationsläufe
- Report-Erzeugung
- Workflow-Ausführung
- Warnungen und Fehler

Die Konsolenausgabe bleibt davon getrennt und dient der direkten Benutzerführung.

Testing

Die Anwendung verwendet pytest.

Aktuelle Tests:

- Aggregationsworkflow
- Bewertungsworkflow

Beispiel:

```
pytest
```

Zusätzlich wurde die Reproduzierbarkeit mit einer frisch erzeugten virtuellen Umgebung und Installation aus `requirements.txt` geprüft.

Der aktuelle Fokus liegt auf fachlichen Workflow-Tests der Logiken für Aggregation und Bewertung. Eine spätere Erweiterung um zusätzliche Unit-Tests für Repository-, Reporting- und Processing-Komponenten ist sinnvoll.

Typischer Ablauf

Datenbank initialisieren

```
python -m app.main init-db
```

Metriken sammeln

```
python -m app.main collect-metrics
```

Tagesworkflow ausführen

```
python -m app.main daily-workflow
```

Monatsaggregation

```
python -m app.main aggregate 2026-05
```

Report erzeugen

```
python -m app.main report 2026-05
```

Gesamter Monatsworkflow

```
python -m app.main monthly-workflow
```

Git-Repository

Der Projektstand ist in einem Git-Repository versioniert.

Repository:

```
https://github.com/Gandolarnier/vm-reporting
```

Nicht versioniert werden:

- `.env`
- `.env.demo`
- virtuelle Umgebung
- SQLite-Datenbanken
- Logdateien
- erzeugte Reports
- IHK-Dokumentation

Stattdessen werden Beispielkonfigurationen bereitgestellt:

- `.env.example`
 - `.env.demo.example`
-

Aktueller Projektstand

Der aktuelle Stand umfasst:

- VMware-vCenter-Anbindung
- historische CPU/RAM-Abfrage
- gastbasierte Storageauswertung
- relationale Persistenz
- CollectionRuns
- Rohdatenhaltung
- Monatsaggregation
- Bewertungslogik
- externe Threshold-Konfiguration
- templatebasiertes HTML-Reporting mit Jinja2
- ausgelagertes CSS
- ausgelagertes JavaScript
- sortierbare Reports
- Inventory Summary
- Summary Cards
- Top Consumers
- Ausweisung von VMs ohne Metrikdaten
- Demo-Modus ohne vCenter-Zugriff
- Debug-Commands
- Service-Schicht
- Workflow-Orchestrierung
- Logging
- Nullable-/None-Fehlertoleranz
- pytest-Tests
- Git-Versionierung
- Automatisierungsvorbereitung
- ausgelagerte Storage-Berechnung in `storage_usage.py`

Die Anwendung befindet sich damit in einem stabilen, nachvollziehbaren und erweiterbaren Zustand.